

Открытая платформа

Открытая платформа

Converged задуман как открытая производственная платформа, а не как закрытая SaaS-коробка. Ядро доступно под open-source лицензией, а вокруг него растут интеграции, микросервисы, микрофронтенды, workflow и прикладные решения.

Для производственного рынка это не идеология, а необходимость. У разных мастерских разное оборудование, разные материалы, разные стандарты качества и разные цепочки поставок. Закрытая система упирается в границы того, что предусмотрел вендор. Открытая позволяет добавить адаптер под конкретный станок или интеграцию с конкретной службой доставки, не дожидаясь чужого релиза.

Расширение может быть маленьким: адаптер оборудования, импорт со старого сайта, отдельный экран, workflow под отраслевой процесс. Если оно полезно другим — попадает в общий реестр. Если нет — остаётся в вашем внедрении и продолжает работать.

Открытость не означает отсутствия контроля. Модуль обязан объявить свою зону ответственности, не ломать архитектурные границы и не получать доступ к данным шире, чем ему нужно. Доверие к экосистеме строится не на обещаниях, а на исходниках, воспроизводимости сборки и явной модели разрешений.

Реестр модулей

Реестр модулей

Реестр — это не отдельный документ и не база данных. Это само дерево исходников.

`modules/` `|— microservices/<домен>/ms-<имя>` домен данных и его API
`|— microfrontends/<домен>/mf-<имя>` экран, подключаемый на лету `|—`
`workflows/wf-<имя>` процесс для DAG-пантайма `|— types/<домен>/ NRPC-`
контракты `|— solutions/` какие модули поставляются вместе

Модуль существует, потому что существует его директория. Он принадлежит домену, потому что лежит в папке этого домена. Он входит в решение, потому что решение называет его в `solutions/solutions.json`. Никакого четвёртого места, где это надо продублировать, нет — и именно поэтому страница экосистемы на сайте собирается обходом дерева, а не редактированием списка.

Назначение модуля берётся из его `README.md` : первый абзац под заголовком `## Purpose` (для микрофронтендов — `## UI Purpose`) и абзац под заголовком о границе ответственности. Эти два абзаца — контракт модуля на человеческом языке, и они обязаны быть в каждом.

Продуктовый слой поверх базы (например, `club`) устроен так же и может опускать уровень домена: его модули лежат прямо в

`modules/microservices/ms-<имя>` . Сборка понимает оба варианта раскладки.

Как добавить модуль

Как добавить модуль

Порядок одинаков для базовой платформы и для продуктового слоя.

1. **Создайте директорию** по соглашению:

`modules/microservices/<домен>/ms-<имя>` для сервиса,
`modules/microfrontends/<домен>/mf-<имя>` для экрана,
`modules/workflows/wf-<имя>` для процесса.

2. **Опишите контракт** в `modules/types/<домен>/` и сгенерируйте клиентов:
`bun run gen`. Клиент появится как пакет `g-<имя>` и станет доступен из браузера, из другого процесса на шине и изнутри workflow.
3. **Напишите README** с разделами `## Purpose` и разделом о границе ответственности. Первые абзацы этих разделов попадут в реестр на сайте — пишите их для читателя, а не для себя.
4. **Включите модуль в решение**, если он поставляется не в одиночку:
добавьте короткое имя в `modules/solutions/solutions.json` и объявите зависимости.
5. **Пересоберите документацию**: `bun run docs` в `core/tools/docs`.
Модуль появится в реестре, счётчики на странице экосистемы пересчитаются сами.

Чего делать не нужно: править списки модулей в данных сайта, дублировать описание в лендинге, регистрировать модуль где-то ещё. Генерация односторонняя — из исходников в данные, и никогда обратно. Всё, что лежит в `data/`, при следующей сборке будет перезаписано.

Требования к модулю на ревью: он не ходит в чужие хранилища, не обходит шину прямыми вызовами, объявляет только те разрешения, которыми пользуется, и не расширяет свою зону ответственности молча.